

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА**

ЗВІТ

ДО ЛАБОРАТОРНОЇ РОБОТИ №1

«Організація професійного Python-проєкту та Git»

з курсу «Професійний Python» Варіант №11 — « Облік замовлень ресторану »

Виконав: Цєньов В.М.

Група: ФЕП-31с

Перевірів: Куньо І.М.

Дата: 30.09.2026

Львів 2026

1. Мета роботи

мета роботи полягає в ознайомленні з основними принципами організації проєкту, створенні ізольованого середовища розробки, формуванні структури за принципом `src-layout`, роботі з модулями, налаштуванні `pyproject.toml` та використанні системи `git`. мій індивідуальний варіант одинадцятий, завдання вимагає створити консольний застосунок для обліку замовлень ресторану. для кожної страви зберігається назва, категорія та ціна, а для замовлення його номер та список страв. реалізовано додавання страв, підрахунок суми, пошук найдорожчої позиції та визначення середньої вартості замовлення.

2. Індивідуальний варіант і постановка задачі --- Варіант 11

Студент має створити додаток для обліку замовлень ресторану виконавши поставлені умови.

Облік замовлень ресторану

Для страви:

- назва;
- категорія;
- ціна.

Для замовлення:

- номер;
- список страв.

Реалізувати:

- створення замовлення;
- додавання страв;
- підрахунок суми;
- найдорожчу позицію;
- середню вартість замовлення.

3. Структура проєкту:

структура проєкту включає файли `pyproject.toml`, `readme.md` та `.gitignore` у корені. далі йде папка `src`, в ній `restaurant_manager`, а там лежать файли `init.py`, `main.py`, `models.py` та `services.py`. проєкт має структуру `src-layout`, де в корені лежать файли конфігурації, а весь вихідний код знаходиться у папці `src` усередині пакета `restaurant_manager`. код розділено на три основні файли. модуль `models.py` містить датакласи, `services.py` відповідає за бізнес-логіку та обчислення, а `main.py` є точкою входу програми.

4. Опис створених модулів

у файлі models.py код виглядає так.

```
from dataclasses import dataclass, field
```

```
@dataclass class dish: name: str category: str price: float
```

```
@dataclass class order: number: int dishes: list[dish] = field(default_factory=list)
```

у файлі services.py прописана бізнес-логіка. from restaurant_manager.models import dish, order

```
def add_dish(order_obj: order, new_dish: dish) -> None:  
    order_obj.dishes.append(new_dish)
```

```
def calc_total(order_obj: order) -> float: return sum(d.price for d in  
    order_obj.dishes)
```

```
def get_most_expensive(order_obj: order) -> dish | None: if not order_obj.dishes:  
    return None return max(order_obj.dishes, key=lambda d: d.price)
```

```
def calc_average_order(orders: list[order]) -> float: if not orders: return 0.0 total =  
    sum(calc_total(o) for o in orders) return total / len(orders)
```

у файлі main.py знаходиться запуск застосунку.

```

from restaurant_manager.models import dish, order
from restaurant_manager.services import add_dish, calc_total, get_most_expensive,
calc_average_order

def main() -> None: d1 = dish("борщ", "перше", 120.0) d2 = dish("стейк",
"основне", 450.0) o1 = order(1)

add_dish(o1, d1)
add_dish(o1, d2)

print("сума замовлення:", calc_total(o1))

expensive = get_most_expensive(o1)
if expensive:
    print("найдорожча страва:", expensive.name)

print("середня вартість усіх замовлень:", calc_average_order([o1]))

if name == "main": main()

```

5. Файл pyproject.toml

файл pyproject.toml визначає метадані та систему збірки проєкту. секція build-system вказує залежність від setuptools, а секція project містить назву restaurant-manager, версію, короткий опис та вимогу python версії 3.11 або вище. секція project.scripts реєструє консольну команду для зручного запуску програми з будь-якого місця. файл .gitignore містить перелік шляхів, які не повинні потрапити до репозиторію, зокрема папку віртуального середовища .venv, директорії кешу **pycache**, компільовані файли з розширенням .pyc, а також папки dist та build. файл readme містить загальний опис проєкту, вказівку про використання python 3.11 та команди для створення віртуального середовища, встановлення пакета і запуску коду через модуль main.

6. Файл .gitignore

```
.venv/  
__pycache__/  
*.pyc  
.pytest_cache/  
.idea/  
.vscode/  
dist/  
build/
```

7. Створення virtual environment та запуск

```
[tsenyov:Lab_01/ (main*) $ python3 -m venv .venv  
[tsenyov:Lab_01/ (main*) $ ls  
[tsenyov:Lab_01/ (main*) $ ls -la  
total 0  
drwxr-xr-x  3 tsenyov  staff   96 Sep 29 23:45 .  
drwxr-xr-x  3 tsenyov  staff   96 Sep 29 23:42 ..  
drwxr-xr-x  7 tsenyov  staff  224 Sep 29 23:45 .venv  
[tsenyov:Lab_01/ (main*) $ source .venv/bin/activate  
(.venv) tsenyov:Lab_01/ (main*) $
```

8. Результат роботи програми

 src — tsenyov@Vitaliis-MacBook-Air — ..on/L

```
[(.venv) tsenyov:src/ (main*) $ python3 main.py  
сума замовлення: 570.0  
найдорожча страва: стейк  
середня вартість усіх замовлень: 570.0  
(.venv) tsenyov:src/ (main*) $
```

9. Робота з Git

```

[(.venv) tsenyov:restaurant_manager/ (main*) $ git commit -m 'complete work'
[main 8b83fb1] complete work
Committer: Vitalii <tsenyov@Vitaliis-MacBook-Air.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

2 files changed, 30 insertions(+)
create mode 100644 python/Lab_01/src/restaurant_manager/models.py
create mode 100644 python/Lab_01/src/restaurant_manager/services.py
[(.venv) tsenyov:restaurant_manager/ (main*) $ git push
[Enter passphrase for key '/Users/tsenyov/.ssh/id_ed25519':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 10 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (8/8), 886 bytes | 886.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:vily70153/Univ.git
   5896ba2..8b83fb1  main -> main
[(.venv) tsenyov:restaurant_manager/ (main*) $ ]

```

10.Відповіді на контрольні запитання

1. що таке python-проект?

організована сукупність коду, конфігурації, документації, тестів та ресурсів.

2. чим проєкт відрізняється від одного скрипта?

проект має модульну структуру, конфігурацію, документацію, тести та керування залежностями.

3. що таке python module?

окремий файл .py, який містить код.

4. що таке python package?

каталог, що об'єднує пов'язані модулі.

5. для чого потрібен **init.py**?

для ініціалізації пакета та, за потреби, визначення його метаданих або api.

6. що таке src-layout?

структура, у якій вихідний код розміщується в окремому каталозі src.

7. які переваги має src-layout?

чіткіша структура та коректніша перевірка встановленого пакета.

8. для чого використовується virtual environment?

для ізоляції залежностей проєкту.

9. як створити .venv?

`python -m venv .venv`

10. як активувати virtual environment?

на windows: `.venv\scripts\activate`; на linux/macos: `source .venv/bin/activate`.

11. для чого використовується pip?

для керування пакетами python.

12. чому рекомендується `python -m pip`?

щоб використовувати pip саме поточного інтерпретатора.

13. для чого потрібен `pyproject.toml`?

для метаданих, системи збірки, залежностей та конфігурації інструментів.

14. що містить секція `[project]`?

назву, версію, опис, вимоги до python, залежності та інші метадані.

15. що таке `editable installation`?

встановлення через `-e`, коли зміни коду одразу доступні пакету.

16. для чого використовується `if name == "main"`?

щоб точка входу виконувалася лише при прямому запуску.

17. що таке git?

розподілена система контролю версій.

18. що таке repository?

сховище файлів проєкту разом з історією git.

19. що таке commit?

зафіксований набір змін з унікальним ідентифікатором.

20. для чого використовується `git status`?

для перегляду стану робочого дерева та індексу.

21. для чого `git add`?

для додавання змін в індекс.

22. для чого `git commit`?

для фіксації підготовлених змін.

- 23.що показує git log?
історію комітів.
- 24.для чого потрібен .gitignore?
щоб git не відстежував зайві або секретні файли.
- 25.чому .venv не потрібно комітити?
воно велике, залежить від платформи і легко відтворюється.
- 26.для чого потрібен readme?
для опису призначення, встановлення та запуску проєкту.
- 27.що таке type hints?
анотації очікуваних типів даних.
- 28.для чого програму розділяють на модулі?
для модульності, повторного використання, тестування та підтримки.
- 29.що таке business logic?
правила та обчислення предметної області.
- 30.чому потрібно відокремлювати точку входу від бізнес-логіки?
щоб логіку було легше тестувати, перевикористовувати та підключати до інших інтерфейсів.

11.Висновок

під час виконання роботи відбувся перехід від створення простих скриптів до професійної організації програмного проєкту. використання модульної архітектури, ізолюваного середовища, системи керування залежностями та системи контролю версій дозволило створити гнучку та надійну кодову базу. цей підхід формує міцний фундамент для наступних етапів розробки, куди можна буде легко інтегрувати складніші структури даних, тестування та додаткові інтерфейси.